

Modern Cryptography and AI/ML Security

Assignment, Vasanth 2026

Maximum Marks: **100**

Course Code: **DSC4110/DSC6111**

Weightage: **20**

Submission Date: **2026-Sep-26**

.....

Instructions:

1. Students can collaborate Maximum 3: Marks will be given as per their clarity and contribution:
 2. Student can meet any day 6:00-7:00 pm at my office for any clarification or help in execution.
 3. Each one have to present in one day:
-

Project 1: Cryptography-Based Secure Model Service

Objective

Design and implement a secure client-server application in which a machine learning model is constructed from a dataset and subsequently provided as a service to authorized users.

The project should demonstrate the practical use of cryptographic techniques for authentication, authorization, confidentiality, integrity, key establishment, and secure communication.

The system consists of three major components:

1. Owner/Admin Application
2. Server Application
3. User Application

The Admin and User applications may be implemented as Android applications, while the Server may be implemented as a web service or REST API.

Model and Application Selection

Students are free to choose any machine learning model and application that they consider relevant to present-day applications.

The choice of model is intentionally left open. For example, students may consider models for:

The primary focus of the project is **not the novelty of the machine learning model**, but the design and implementation of a **secure model-serving system using cryptographic techniques**.

Students should clearly justify their choice of:

1. Application
2. Dataset
3. Machine learning model
4. Model-serving scenario

1. System Model

Let D be the selected training dataset and M be the resulting machine learning model:

$$\boxed{\text{BuildModel}(D) \rightarrow M}$$

The Owner deploys M on the Server. An authorized user submits an input x and receives the model output y :

$$\boxed{\text{Query}(x) \rightarrow y}$$

New data D' collected through the system may be used to update the model:

$$\boxed{\text{RebuildModel}(D, D') \rightarrow M'}$$

The updated model M' can then be securely deployed on the Server.

The choice of dataset, application, and machine learning model is open to the students. However, the **cryptographic design and security of the complete system are mandatory components of the project.**

2. System Entities

Owner / Admin

The Owner is responsible for managing the model, users, and collected data.

The Owner should support the following operations:

- $\text{BuildModel}(D) \rightarrow M$: Construct a model from the initial dataset D .
- $\text{DeployModel}(M)$: Deploy the model on the Server.
- $\text{ApproveUser}(UID)$: Approve a user's registration request.
- $\text{ViewNewData}()$: View data collected from authorized users.
- $\text{DownloadNewData}() \rightarrow D'$: Download newly collected data.
- $\text{RebuildModel}(D, D') \rightarrow M'$: Construct an updated model using the original and newly collected datasets.
- $\text{RemoveUser}(UID)$: Remove or revoke a user's access.

Server

The Server acts as the central service provider and processes requests from the Owner and Users.

The Server should support operations such as:

- $\text{RequestForUserRegistration}()$: Receive registration requests from users.
- $\text{RequestForApproval}(UID)$: Maintain or forward pending user approval requests.
- $\text{UpdateModel}(M')$: Replace the currently deployed model M with the new model M' after authorization from the Owner.
- $\text{Query}(x) \rightarrow y$: Execute the deployed model on input x and return the result y to an authorized user.
- $\text{StoreFeedback}(UID, x, y, f)$: Store feedback associated with a user's query and the returned response.

User

A User interacts with the system through the User Application.

The User should support the following operations:

- `RequestForRegistration()`: Submit a registration request to the Server.
- `RequestForLogin()`: Authenticate with the Server.
- `RequestForQuery( $x$ )  $\rightarrow y$` : Submit a data point x and obtain the corresponding model output y .
- `SubmitFeedback( $f$ )`: Provide feedback or a rating for the returned result.

3. Secure Communication Requirements

The communication between any two entities in the system must be protected using a secure channel.

Assumptions

1. Every entity i has a public/private key pair:

$$(pk_i, sk_i).$$

2. The public key of every entity is available to the other entities.
3. Students do not need to implement or verify digital certificates. Certificate management and PKI are outside the scope of this project.
4. Public/private key pairs are assumed to be generated and stored securely.
5. The communication network is considered insecure.
6. An attacker may intercept, modify, replay, or drop network messages.

4. Secure Channel Establishment

Before every communication session between two parties, a fresh secure channel must be established.

Suppose two parties A and B want to communicate. Their public/private key pairs are

$$(pk_A, sk_A) \quad \text{and} \quad (pk_B, sk_B).$$

The secure channel consists of the following two main steps.

Step 1: KEM-Based Key Establishment

Party A uses the public key of B to establish a shared secret using a Key Encapsulation Mechanism (KEM):

$$(ct, K) \leftarrow \text{Encaps}(pk_B).$$

Party A sends the ciphertext ct to B :

$$A \longrightarrow B : ct$$

Party B uses its private key to recover the shared secret:

$$K \leftarrow \text{Decaps}(sk_B, ct).$$

After successful decapsulation,

$$\boxed{K_A = K_B = K}.$$

The shared secret K must be fresh for every new communication session.

Step 2: Symmetric-Key Communication

The shared secret obtained from the KEM should be used to establish symmetric session keys.

Students should use an appropriate Key Derivation Function (KDF):

$$K_{\text{enc}} = \text{KDF}(K, \text{context}).$$

The resulting session key is then used for symmetric-key communication.

For example,

$$c \leftarrow \text{Encrypt}(K_{\text{enc}}, m)$$

and

$$m \leftarrow \text{Decrypt}(K_{\text{enc}}, c).$$

Students may use a suitable block cipher or stream cipher. The use of authenticated encryption is strongly recommended.

5. Protocol at a Glance

The overall cryptographic pipeline should follow:

$\text{KEM} \longrightarrow \text{KDF} \longrightarrow \text{Session Key} \longrightarrow \text{Symmetric Encryption}$
--

For a communication session between A and B :

$$\begin{aligned} A & : (ct, K) \leftarrow \text{Encaps}(pk_B) \\ A & \longrightarrow B : ct \\ B & : K \leftarrow \text{Decaps}(sk_B, ct) \\ A, B & : K_{\text{enc}} \leftarrow \text{KDF}(K, \text{context}) \\ A & \longrightarrow B : \text{Enc}_{K_{\text{enc}}}(m) \\ B & \longrightarrow A : \text{Enc}_{K_{\text{enc}}}(m') \end{aligned}$$

Thus, public-key cryptography is used for establishing a fresh shared secret, while subsequent communication uses efficient symmetric-key cryptography.

6. Required Security Properties

The implementation must provide the following security properties:

1. **Confidentiality:** An adversary observing network traffic should not be able to recover the plaintext.
2. **Integrity:** Any modification of a transmitted ciphertext must be detected.
3. **Fresh Session Keys:** A new shared secret must be generated for every communication session.
4. **Replay Protection:** Replaying a previously captured message must not result in the message being accepted as a new communication.
5. **Secure Key Derivation:** Session keys should be derived using an appropriate KDF.
6. **Secure Nonce/IV Management:** Nonces or initialization vectors must be generated and managed according to the requirements of the selected symmetric encryption scheme.

7. **Session Termination:** Session keys should no longer be used after the corresponding communication session has been terminated.

7. Implementation Requirements

Students must implement the complete cryptographic protocol rather than simply using an existing secure communication protocol as a black box.

The implementation and report must clearly specify:

1. The selected KEM and its parameters.
2. The selected symmetric encryption algorithm.
3. The selected KDF.
4. Key-generation procedures.
5. Secure session-establishment protocol.
6. Message encryption and decryption.
7. Nonce/IV generation and management.
8. Session termination and key deletion.
9. Threat model.
10. Security analysis.

8. Expected Deliverables

Each group must submit:

1. Owner/Admin Application.
2. User Application.
3. Server Application / API.
4. Database.
5. Cryptographic Protocol Specification.
6. Threat Model and Security Analysis.
7. Source Code.
8. Final Project Report.
9. Final Demonstration.